

Data Visualization with **seaborn**

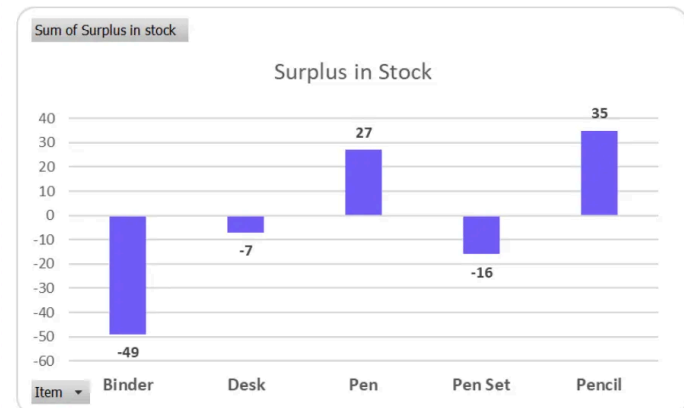
Eunji Kim

eunjikim@cau.ac.kr

A picture is worth a thousand words.

Order Date	Region	Employee ID	Name	Surname	Full Name	Item	Units In Inventory	Sold Units	Surplus in stock	Unit Cost	Total
1/4/21	East	11970	David Jones	Jones	David Jones	Pencil	100	93	5	1.99	189.01
1/23/21	Central	401971	John Kivell	Kivell	John Kivell	Binder	60	50	10	19.99	999.50
2/9/21	Central	281048	Gabriel Jardine	Jardine	Gabriel Jardine	Pencil	36	36	0	4.99	179.64
2/26/21	Central	127572	John Gill	Gill	John Gill	Pen	27	27	0	19.99	539.73
3/15/21	West	44643	Andreas Sorvino	Sorvino	Andreas Sorvino	Pencil	56	56	0	2.99	167.44
4/1/21	East	324744	Bridget Jones	Jones	Bridget Jones	Binder	10	60	-50	4.99	259.40
4/18/21	Central	229150	Rob Andrews	Andrews	Rob Andrews	Pencil	75	75	0	1.99	149.25
5/5/21	Central	339307	Gabriel Jardine	Jardine	Gabriel Jardine	Pencil	90	90	0	4.99	449.10
5/22/21	West	38004	Dave Thompson	Thompson	Dave Thompson	Pencil	32	32	0	1.99	63.68
6/8/21	East	139708	Bridget Jones	Jones	Bridget Jones	Binder	60	60	0	8.99	539.40
6/25/21	Central	370024	TJ Morgan	Morgan	TJ Morgan	Pencil	90	90	0	4.99	449.10
7/12/21	East	309647	Len Howard	Howard	Len Howard	Binder	29	29	0	1.99	57.71
7/29/21	East	380608	Steve Parent	Parent	Steve Parent	Binder	81	81	0	19.99	1,619.19
8/15/21	East	452988	Bridget Jones	Jones	Bridget Jones	Pencil	35	35	0	4.99	174.65
9/1/21	Central	264816	Tom Smith	Smith	Tom Smith	Desk	2	2	0	125.00	250.00
9/18/21	East	143055	Bridget Jones	Jones	Bridget Jones	Pen Set	16	16	0	15.99	255.84
10/5/21	Central	426184	TJ Morgan	Morgan	TJ Morgan	Binder	70	28	42	8.99	251.72
10/22/21	East	139708	Bridget Jones	Jones	Bridget Jones	Pen	64	64	0	8.99	575.36
11/8/21	East	99377	Steve Parent	Parent	Steve Parent	Pen	15	15	0	19.99	299.85
11/25/21	Central	401971	John Kivell	Kivell	John Kivell	Pen Set	57	53	4	4.99	264.47
12/12/21	Central	56018	Jonathan Smith	Smith	Jonathan Smith	Pencil	11	89	-78	1.29	114.81
12/29/21	East	88260	Steve Parent	Parent	Steve Parent	Pen Set	8	23	-15	15.99	367.77
1/15/22	Central	65626	John Gill	Gill	John Gill	Binder	75	31	44	8.99	278.69
2/1/22	Central	474696	Steve Smith	Smith	Steve Smith	Binder	88	50	18	15.00	750.00
2/18/22	East	139708	John Jones	Jones	John Jones	Binder	79	55	23	4.99	274.45
3/7/22	West	426013	Jonathan Sorvino	Sorvino	Jonathan Sorvino	Binder	52	48	4	19.99	959.52
3/24/22	Central	482378	Steve Jardine	Jardine	Steve Jardine	Pen Set	15	1	14	4.99	4.99
4/10/22	Central	229150	John Andrews	Andrews	John Andrews	Pencil	84	13	71	1.99	25.87
4/27/22	East	231018	Steve Howard	Howard	Steve Howard	Pen	31	57	-26	4.99	284.43
5/14/22	Central	436791	John Gill	Gill	John Gill	Pencil	40	86	-46	1.29	110.94
5/31/22	Central	436791	Jonathan Gill	Gill	Jonathan Gill	Binder	85	42	43	8.99	377.58
6/17/22	Central	401971	Steve Kivell	Kivell	Steve Kivell	Desk	29	3	26	125.00	375.00
7/4/22	East	468311	John Jones	Jones	John Jones	Pen Set	10	65	-55	4.99	324.35
7/21/22	Central	400131	Steve Morgan	Morgan	Steve Morgan	Pen Set	62	54	8	12.49	674.46
8/7/22	Central	401971	John Kivell	Kivell	John Kivell	Pen Set	60	27	33	23.95	646.65
8/24/22	West	262588	Jonathan Sorvino	Sorvino	Jonathan Sorvino	Desk	53	55	-2	275.00	15,125.00
9/10/22	Central	436791	Steve Gill	Gill	Steve Gill	Pencil	63	88	-25	1.29	113.52

VS.



Data in table

Data visualized in chart

Matplotlib and Seaborn

- Two most popular packages for data visualization
- Seaborn provides a high-level interface to matplotlib and makes it easier to produce graphs.

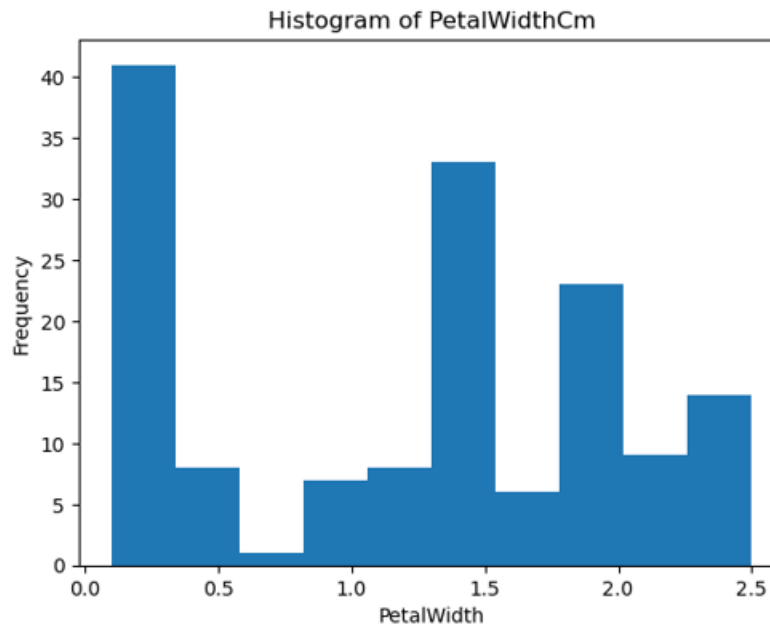


Draw a histogram using matplotlib

Draw a histogram of PetalWidthCm using matplotlib

```
import matplotlib.pyplot as plt
```

```
plt.hist(df['PetalWidthCm'], bins=10) # number of bins: 10  
plt.xlabel('PetalWidth') # show labels of x-axis explicitly for a better delivery  
plt.ylabel('Frequency') # show labels of y-axis explicitly for a better delivery  
plt.title('Histogram of PetalWidthCm')
```

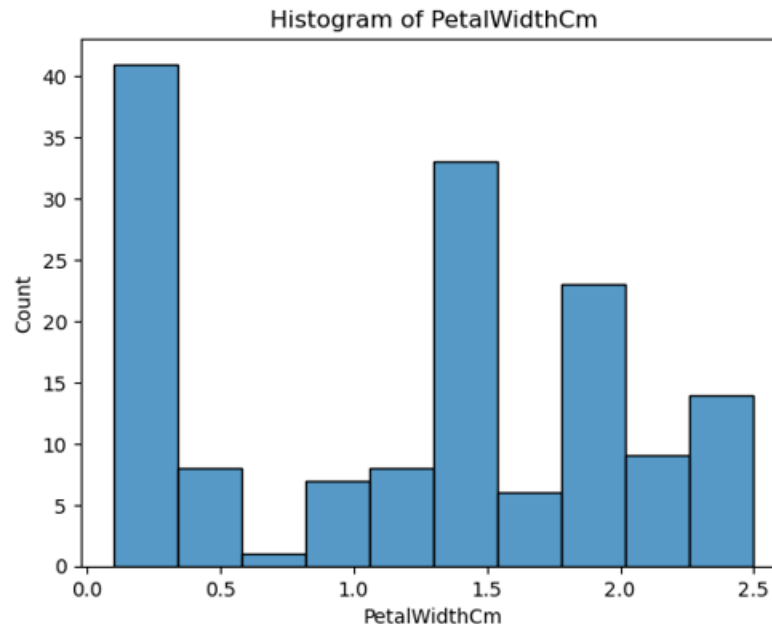


Draw a histogram using seaborn

Draw a histogram of PetalWidthCm using seaborn

```
import seaborn as sns

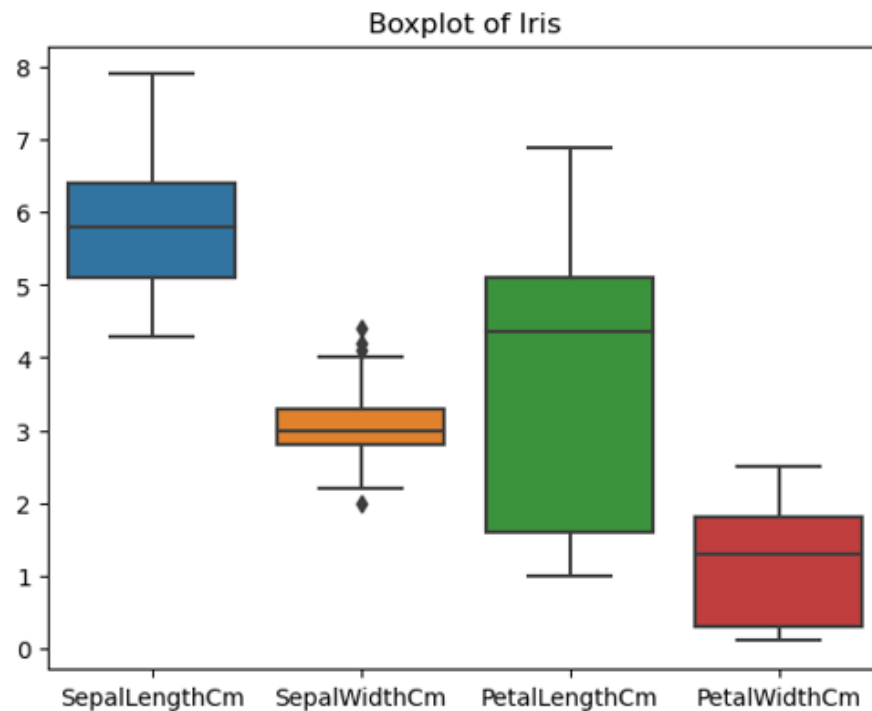
sns.histplot(
    df['PetalWidthCm'],
    bins=10
).set(title='Histogram of PetalWidthCm')
```



Boxplot using seaborn

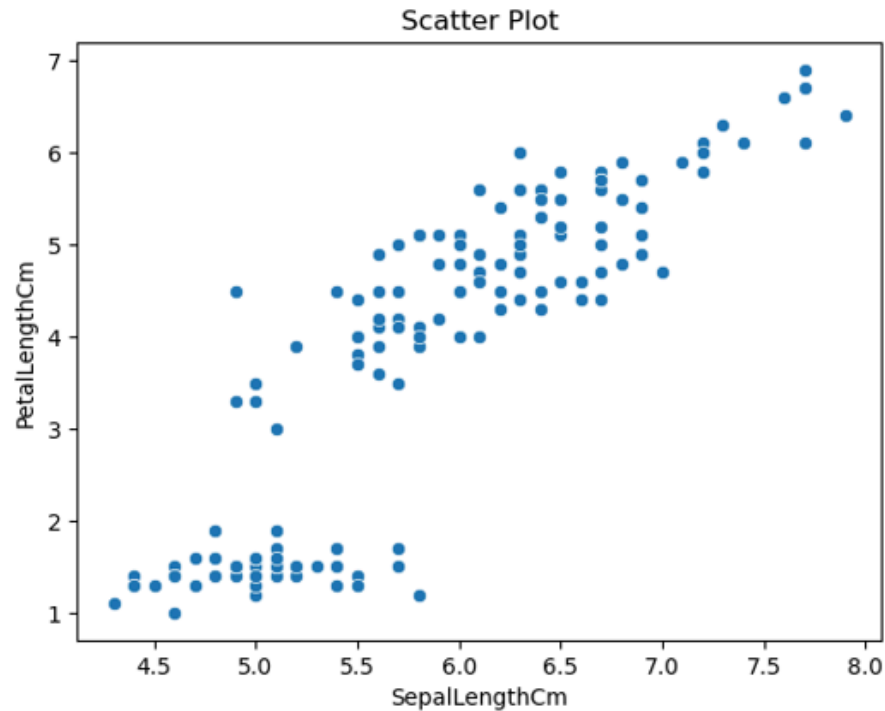
Efficient to compare distributions of multiple attributes

```
df_num = df.iloc[:, :-1]  
sns.boxplot(df_num).set(title='Boxplot of Iris')
```



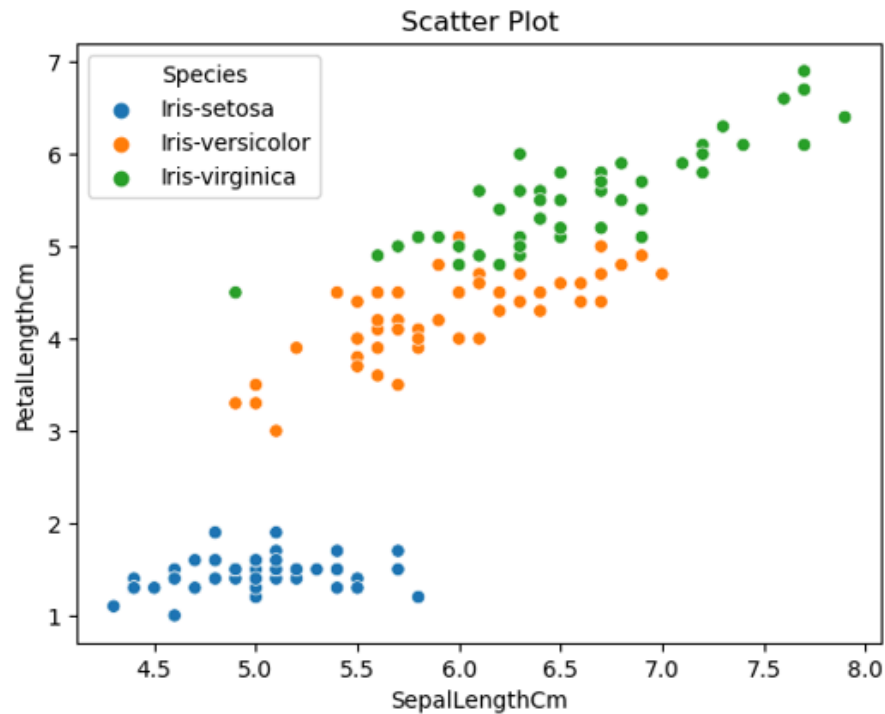
Scatterplot using seaborn

```
sns.scatterplot(  
    x=df['SepalLengthCm'],  
    y=df['PetalLengthCm']  
).set(title="Scatter Plot")
```



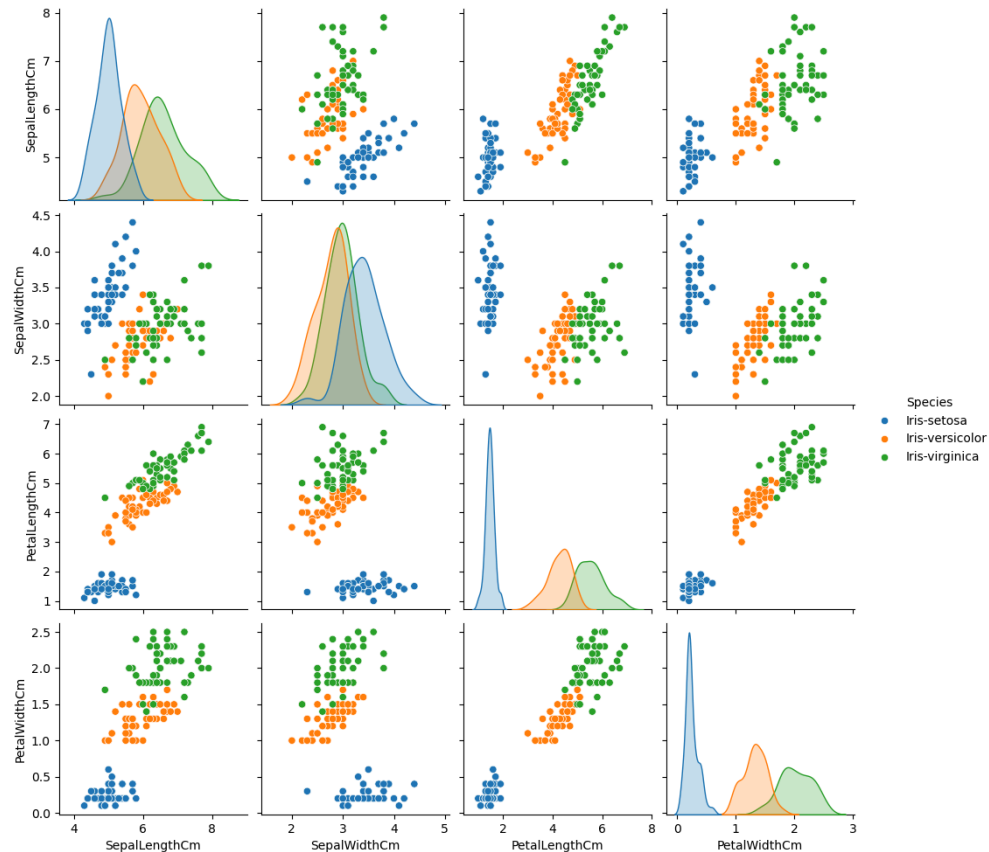
Add hue by species:

```
sns.scatterplot(  
    x=df['SepalLengthCm'],  
    y=df['PetalLengthCm'],  
    hue=df['Species']  
).set(title="Scatter Plot")
```



Scattermatrix using seaborn

```
sns.pairplot(df, hue='Species')
```



Save the Figure

Save the current figure as `myfigure.png` in the current working directory:

```
# import matplotlib.pyplot as plt

# # Example plot
# plt.plot([1, 2, 3], [4, 5, 6])
# plt.title("Example Figure")

# save the figure
plt.savefig('./myfigure.png')
```

Seaborn Example Gallery

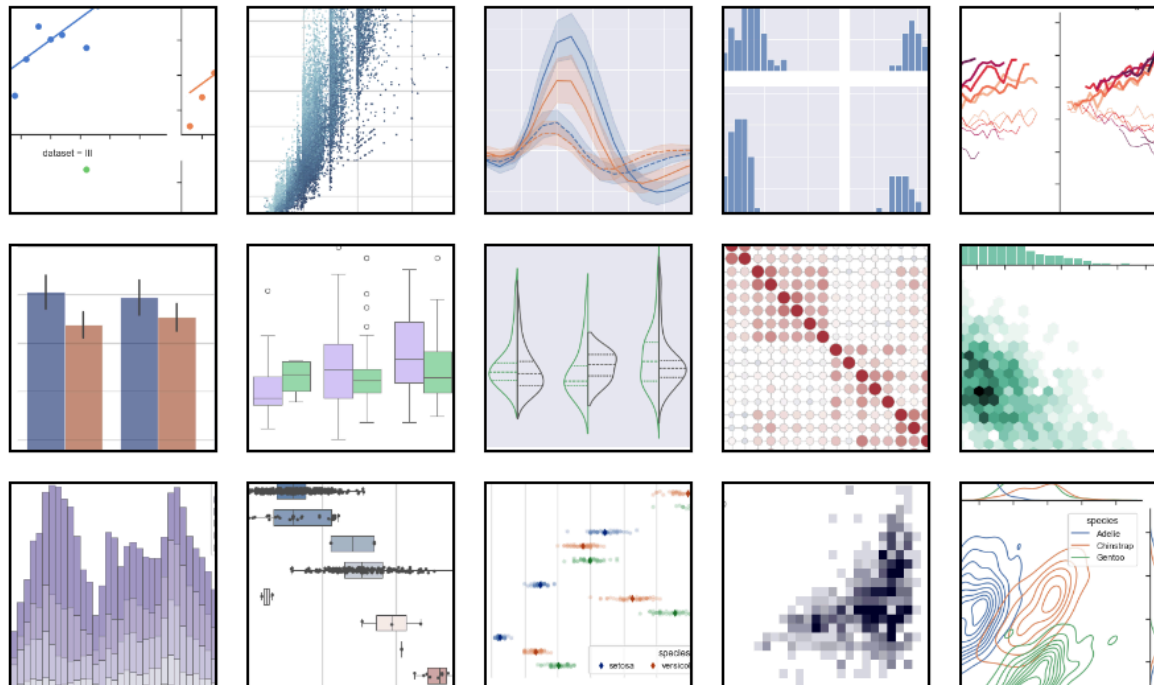
You can explore Seaborn examples and their corresponding code at:

<https://seaborn.pydata.org/examples/>



Installing **Gallery** Tutorial API Releases Citing FAQ

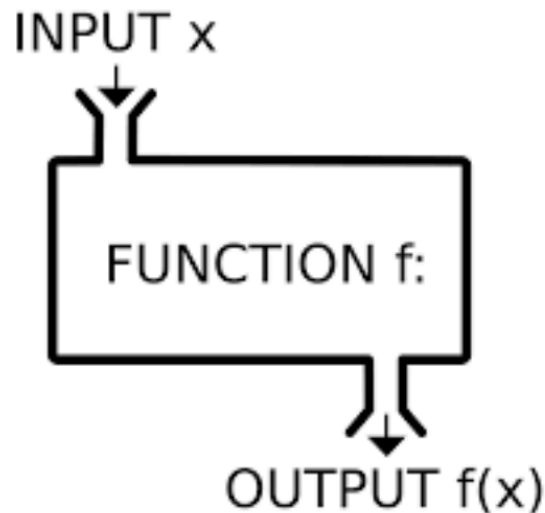
Example gallery



Develop a Classification "Model"

Let's Build a Classification "Model"

We've explored the data. Now, let's use our insights to make predictions!



Our Goal:

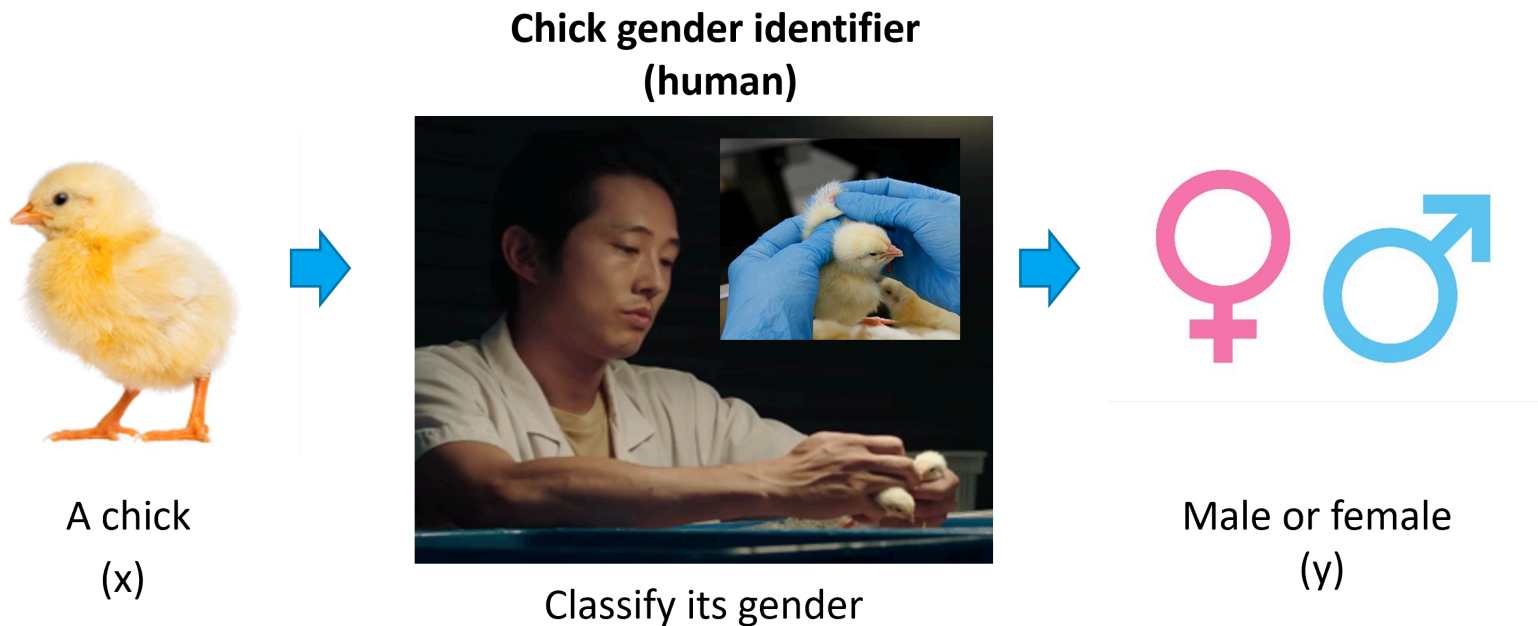
Can we build a function (f) to classify the Species (y) of an iris, just by looking at its measurements (x)?

This process is called **Classification**.

Example: Chick gender classification (by Human)

Classification is the process of categorizing items into predefined categories or **classes**.

- Input (x): several features of a chick (feather length, wing size, ...)
- Output (y): chick's gender



Example: Chick gender classification (by Machine)

Classification is the process of categorizing items into predefined categories or **classes**.

- Input (x): several features of a chick (feather length, wing size, ...)
- Output (y): chick's gender



We can make our computer to classify Iris flower into one of the predefined classes!

Components of Classification

- Input (`x` , Features): The measurements we have (e.g., `PetalWidthCm`).
- Model (`f` , Our Rule): A function we will write.
- Output (`y` , Class): The "Category" we predict (e.g., `setosa` , `versicolor` , `virginica`).

We will use `.apply()` to build our model!

"Ground Truth" vs. "Prediction"

To see if our model is any good, we need to compare its guesses to the *correct* answers.

- **Ground Truth (or True Label):**

- The *correct* answer. The "answer key."
- In our dataset, the `Species` column is our **Ground Truth**. A human expert has already labeled these for us.

- **Prediction:**

- The model's *guess*.
- We will create a **new column** (e.g., `MyPrediction`) to store our model's guesses.

Our goal is to make our `MyPrediction` column match the `Species` column as closely as possible.

Our "Models" Will Be Functions

A "model" sounds complicated, but for us, it will just be a Python function that contains our rules.

We will write two types of models:

1. Univariate Model:

- Uses only **one feature** to make a decision.
- *Example Rule:* IF PetalLengthCm < 2.5 THEN predict 'Iris-setosa'

2. Multivariate Model:

- Uses **two or more features** to make a decision.
- *Example Rule:* IF PetalLengthCm < 4.8 AND PetalWidthCm < 1.5 THEN predict 'Iris-versicolor'

Goodness of the Model: Accuracy

The most common way to measure performance is **Accuracy**.

Formula:

$$\text{Accuracy} = (\text{Number of Correct Guesses}) / (\text{Total Number of Guesses})$$

- Accuracy is just like a test score.
- It measures what *fraction* of the guesses our model got right.

Example:

- Our dataset has 150 rows.
- Our model makes its predictions.
- We compare our predictions to the "Ground Truth" (`Species`) column and find we got **141** correct.
- **Accuracy = 141 / 150 = 0.94 (or 94%)**

Thank You.